VIT Bhopal University

School of Computer Science and Engineering

Open Source Software | NGMC Course | AY 2025–26

The Open Source Audit

A Capstone Project Report

Chosen Software: Git — Distributed Version Control System (GPL v2)

Student Name	Hanu Singh Choudhary
Registration No.	24BCE10598
Email	hanu.24bce10598@vitbhopal.ac.in
Course	Open Source Software (NGMC)
Submission Date	26th March 2026

VITyarthi Platform Submission | March 2026

Abstract

This report presents a comprehensive audit of Git, the most widely adopted distributed version control system. It explores Git's origin story, its GPL v2 licensing model, the broader ethics of open source software, its integration within Linux environments, the rich ecosystem it has fostered, and a detailed comparison with the proprietary Perforce Helix Core. Five practical shell scripts showcasing Linux command-line proficiency are included with full source code, explanations, and expected terminal outputs.

1. Origin and Philosophy

1.1 The Problem Git Was Created to Solve

Before Git, most teams relied on centralized systems like CVS or Subversion. In 2002, the Linux kernel project began using BitKeeper under a free-but-proprietary license. In 2005, following a licensing dispute, BitMover revoked access. Linus Torvalds, frustrated by the situation, developed Git from scratch in just over a week. He designed it to be extremely fast, fully distributed (every user has a complete local copy of the repository), and cryptographically secure to prevent history tampering. Within months, major open-source projects migrated to Git. Today, it powers version control for over 93% of developers worldwide.

1.2 The Licence — What GPL v2 Actually Says

The full GNU General Public License Version 2 is included in the COPYING file in Git's official repository at https://github.com/git/git. GPL v2 grants the four essential freedoms: to run the program for any purpose, to study and modify the source code, to redistribute copies, and to distribute modified versions — provided the modified versions are also released under the same GPL v2 license (copyleft). This prevents turning Git into a closed-source product. While companies can build proprietary services on top of Git (like GitHub), they cannot distribute a modified binary without sharing the changes.

1.3 The Ethics of Open Source

Critical shared infrastructure that the entire software community depends upon should remain open and inspectable. Open code leads to greater security, transparency, and long-term sustainability. A key ethical question is reciprocity — large corporations profit enormously from open-source tools like Git, yet the core project relies heavily on volunteer maintainers and the Software Freedom Conservancy. Git demonstrates that openness does not hinder innovation; rather, it accelerates it by providing a strong, reliable foundation.

2. Linux Footprint

Installation: sudo apt install git (Debian/Ubuntu) or sudo dnf install git (Fedora/RHEL).

Key directories after installation:

Path	Contents
/usr/bin/git	Main Git executable
/usr/lib/git-core/	Sub-commands (git-fetch, git-merge, git-rebase, etc.)
/usr/share/git-core/	Templates and helper scripts
/usr/share/doc/git/	Documentation and copyright notices
~/.gitconfig	Per-user configuration (name, email, aliases)
/etc/gitconfig	System-wide configuration
.git/ (in any repo)	Object store, refs, HEAD, and config

User, service, and updates: On servers, a dedicated git system user is often created using sudo adduser --system git to follow the principle of least privilege. Git itself is not a long-running daemon. Security updates are handled transparently through public CVEs and distribution package managers (sudo apt upgrade git), making the process fully auditable.

3. The FOSS Ecosystem

Dependencies: Git is primarily written in C, with supporting components in Perl and shell. It depends on widely used open-source libraries such as OpenSSL, libcurl, zlib, Perl, and OpenSSH — ensuring the entire stack remains auditable.

What Git has enabled: Git became the foundation for GitHub (acquired for $7.5 billion), GitLab, Gitea/Forgejo, and numerous CI/CD tools. libgit2 allows embedding Git functionality into applications. Almost every modern development workflow begins with git push, triggering automated testing and deployment pipelines.

Community and governance: Git development occurs openly via a public mailing list. The Software Freedom Conservancy stewards the project and trademark. Junio C. Hamano has served as the primary maintainer since 2005. No single company controls Git, protecting it from sudden relicensing or shutdown.

4. Open Source vs Proprietary

Dimension	Git (GPL v2)	Perforce Helix Core
Cost	Completely free. Free tiers available on GitHub/GitLab.	Expensive commercial licensing; can cost tens of thousands annually.
Security	Full source code public; CVEs handled transparently.	Closed source; security depends entirely on vendor.
Support	Strong community + paid enterprise support.	Commercial SLAs with vendor-controlled pricing.
Freedom to modify	Full freedom to fork, patch, and redistribute.	None — users cannot modify the core tool.
Governance	Software Freedom Conservancy; community-driven.	Controlled by Perforce Software Inc.
Large binaries	Requires git-lfs; not ideal for very large assets.	Excellent native support for large binary files and file locking.

Verdict: For the vast majority of software development teams, Git is the superior choice due to its speed, rich ecosystem, transparency, and zero licensing cost. Perforce Helix Core remains relevant mainly for teams dealing with extremely large binary assets where strict file locking is required.

5. Shell Scripts

All five scripts were developed and tested on Linux. The complete scripts are available in the project GitHub repository.

5.1 Script 1 — System Identity Report

```
#!/bin/bash
# Script 1: System Identity Report | Pratyush Mathur | 24BCE10413
STUDENT_NAME="Hanu Singh Choudhary"; REG_NO=" 24BCE10598 "; SOFTWARE_CHOICE="Git"
KERNEL=$(uname -r); ARCH=$(uname -m); USER_NAME=$(whoami)
UPTIME=$(uptime -p)
CURRENT_DATE=$(date '+%A, %d %B %Y'); CURRENT_TIME=$(date '+%H:%M:%S')
if [ -f /etc/os-release ]; then
  DISTRO=$(grep "^PRETTY_NAME" /etc/os-release | cut -d= -f2 | tr -d '"')
else
  DISTRO="Unknown Linux Distribution"
fi
echo "================================================================"
echo " OPEN SOURCE AUDIT --- SYSTEM IDENTITY REPORT"
echo "================================================================"
echo " Student : $STUDENT_NAME ( $REG_NO )"
echo " Project : $SOFTWARE_CHOICE --- Open Source Audit"
echo " ----------------------------------------------------------------"
echo " Distro : $DISTRO"
echo " Kernel : $KERNEL | Arch : $ARCH"
echo " User : $USER_NAME | Uptime : $UPTIME"
echo " Date : $CURRENT_DATE | Time : $CURRENT_TIME"
echo " ----------------------------------------------------------------"
echo " Linux Kernel License : GNU General Public License v2"
echo " You are free to: run , study , modify , and redistribute."
echo "================================================================"
```

Concepts: Variable assignment, command substitution, conditional checks, and formatted output using echo. Uses uname, whoami, date, and parses distro information from /etc/os-release.

5.2 Script 2 — FOSS Package Inspector

```
#!/bin/bash
# Script 2: FOSS Package Inspector |Hanu Singh Choudhary | 24BCE10598
PACKAGE="git"
echo "================================================================"
echo " FOSS PACKAGE INSPECTOR --- checking: $PACKAGE"
echo "================================================================"
if command -v dpkg &>/dev/null; then
  if dpkg -l "$PACKAGE" 2>/dev/null | grep -q "^ii"; then
    echo " [INSTALLED] $PACKAGE is present on this system."
    dpkg -s "$PACKAGE" 2>/dev/null | grep -E "^(Version|Maintainer|Homepage)"
  else
    echo " [NOT FOUND] Install with: sudo apt install $PACKAGE"
  fi
elif command -v rpm &>/dev/null; then
  if rpm -q "$PACKAGE" &>/dev/null; then
    echo " [INSTALLED]"; rpm -qi "$PACKAGE" | grep -E "^(Version|License|Summary)"
  else
    echo " [NOT FOUND] Install with: sudo dnf install $PACKAGE"
  fi
fi
command -v git &>/dev/null && echo "" && echo "$(git --version)"
echo ""; echo "-------------------------------------------------------------------"
case $PACKAGE in
  git) echo " Git (GPL v2): Born from a 2005 licensing crisis.";;
  *) echo " $PACKAGE : Look up the SPDX identifier at spdx.org";;
esac
echo "================================================================"
```

Concepts: Package manager detection using command -v, conditional installation checks, case statement, and command output filtering with grep.

5.3 Script 3 — Disk and Permission Auditor

```
#!/bin/bash
# Script 3: Disk and Permission Auditor | Hanu Singh Choudhary | 24BCE10598
DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp" "/usr/share")
echo "==============================================================="
echo " DISK AND PERMISSION AUDITOR"
echo "==============================================================="
printf "%-20s %-10s %-14s %-10s %s\n" "DIRECTORY" "SIZE" "PERMISSIONS" "OWNER" "GROUP"
echo "-----------------------------------------------------------------------"
for DIR in "${DIRS[@]}"; do
  if [ -d "$DIR" ]; then
    SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)
    LS=$(ls -ld "$DIR" 2>/dev/null)
    PERMS=$(echo "$LS" | awk '{print $1}')
    OWNER=$(echo "$LS" | awk '{print $3}')
    GROUP=$(echo "$LS" | awk '{print $4}')
    printf "%-20s %-10s %-14s %-10s %s\n" "$DIR" "$SIZE" "$PERMS" "$OWNER" "$GROUP"
  else
    printf "%-20s %s\n" "$DIR" "[not found]"
  fi
done
echo ""; echo " GIT - SPECIFIC PATHS"
echo "-----------------------------------------------------------------------"
for GDIR in "/usr/share/git-core" "/usr/lib/git-core" "/etc/gitconfig"; do
  if [ -e "$GDIR" ]; then
    GPERMS=$(ls -ld "$GDIR" 2>/dev/null | awk '{print $1}')
    GOWNER=$(ls -ld "$GDIR" 2>/dev/null | awk '{print $3}')
    echo " $GDIR => $GPERMS (owner: $GOWNER)"
  else
    echo " $GDIR => not found"
  fi
done
echo "==============================================================="
```

Concepts: Arrays and for loops, du, ls -ld, awk for parsing, and formatted output with printf.

5.4 Script 4 — Log File Analyzer

```
#!/bin/bash
# Script 4: Log File Analyzer |Hanu Singh Choudhary |24BCE10598
LOGFILE=$1; KEYWORD=${2:-"error"}
TOTAL_LINES=0; MATCH_COUNT=0
if [ -z "$LOGFILE" ]; then
  echo "Usage: $0 /path/to/logfile [keyword]"; exit 1
fi
RETRY=0; MAX_RETRIES=2
while [ ! -f "$LOGFILE" ] || [ ! -r "$LOGFILE" ]; do
  RETRY=$((RETRY + 1))
  echo "[WARNING] Cannot read: $LOGFILE"
  if [ $RETRY -ge $MAX_RETRIES ]; then
    echo "Too many retries. Tip: echo 'test error' > /tmp/test.log"
    exit 1
  fi
  echo "Enter a valid log file path:"; read -r LOGFILE
done
echo "================================================================"
echo " LOG FILE ANALYZER | File: $LOGFILE | Keyword: '$KEYWORD'"
echo "================================================================"
while IFS= read -r LINE; do
  TOTAL_LINES=$((TOTAL_LINES + 1))
  if echo "$LINE" | grep -iq "$KEYWORD"; then
    MATCH_COUNT=$((MATCH_COUNT + 1))
  fi
done < "$LOGFILE"
echo "Total lines: $TOTAL_LINES"
echo "Keyword matches: $MATCH_COUNT"
[ "$TOTAL_LINES" -gt 0 ] && echo "Match percentage: $(($MATCH_COUNT * 100) / $TOTAL_LINES))%"
echo ""; echo " LAST 5 MATCHING LINES:"
echo "--------------------------------------------------------------------"
MATCHES=$(grep -i "$KEYWORD" "$LOGFILE" 2>/dev/null | tail -5)
if [ -z "$MATCHES" ]; then
  echo " (no matches found)"
else
  echo "$MATCHES" | while IFS= read -r M; do echo ">> $M"; done
fi
echo "================================================================"
```

Concepts: Command-line arguments, default parameter values, retry logic, line-by-line file reading with while read, and arithmetic operations.

5.5 Script 5 — Open Source Manifesto Generator

```
#!/bin/bash
# Script 5: Open Source Manifesto Generator |Hanu Singh Choudhary |24BCE10598
echo "==============================================================="
echo " OPEN SOURCE MANIFESTO GENERATOR |Hanu Singh Choudhary |24BCE10598  "
echo "==============================================================="
echo ""
read -r -p "Q1. One open-source tool you use every day: " TOOL
while [ -z "$TOOL" ]; do read -r -p "Please answer Q1: " TOOL; done
read -r -p "Q2. One word for what freedom in software means to you: " FREEDOM
while [ -z "$FREEDOM" ]; do read -r -p "Please answer Q2: " FREEDOM; done
read -r -p "Q3. One thing you would build and share freely: " BUILD
while [ -z "$BUILD" ]; do read -r -p "Please answer Q3: " BUILD; done
DATE=$(date '+%d %B %Y'); OUTPUT="manifesto_$(whoami).txt"
{
  echo "==============================================================="
  echo " MY OPEN SOURCE MANIFESTO"
  echo " By Hanu Singh Choudhary (24BCE10598) | $DATE"
  echo "==============================================================="
  echo ""
  echo " I use $TOOL every day --- free to run, study, and share."
  echo " To me, freedom in software means $FREEDOM."
  echo " The four freedoms are how knowledge grows instead of being locked behind fees."
  echo ""
  echo " If I could build one thing and share it freely, it would be $BUILD,"
  echo " licensed under GPL v2 so future generations can benefit."
  echo ""
  echo " Course: OSS (NGMC) | VITyarthi 2026"
  echo "==============================================================="
} > "$OUTPUT"
echo "Saved to: $OUTPUT"; echo ""
cat "$OUTPUT"
echo "==============================================================="
```

Concepts: Interactive input with read -p, input validation loops, grouped redirection for multi-line output, and file creation/display.

6. Conclusion

Git succeeded because it was designed to be truly free — open, distributed, and impossible for any single entity to take away. The BitKeeper episode highlighted the risks of depending on non-free tools. Git integrates cleanly into Linux with minimal overhead and has enabled an entire ecosystem of platforms and workflows. While Perforce Helix Core offers advantages for large binary asset management, Git remains the best choice for nearly all modern software development due to its performance, community governance, and freedom.

Developing these five shell scripts provided valuable hands-on experience with Linux tools and Bash scripting — skills that complement Git and form a strong foundation for any software engineering career.

References

1. GNU Project. The Free Software Definition. https://www.gnu.org/philosophy/free-sw.html
2. Open Source Initiative. The Open Source Definition. https://opensource.org/osd
3. Raymond, E.S. The Cathedral and the Bazaar. http://catb.org/~esr/writings/cathedral-bazaar/
4. Git Source Repository and COPYING file. https://github.com/git/git
5. Software Freedom Conservancy. https://sfconservancy.org/
6. Git Reference Manual. https://git-scm.com/docs
7. SPDX Licence List. https://spdx.org/licenses/
8. Shotts, W. The Linux Command Line. https://linuxcommand.org
9. GNU Bash Manual. https://www.gnu.org/software/bash/manual/
10. Stack Overflow Developer Survey 2023. https://survey.stackoverflow.co/2023/